

Date 17th September 2025

Design And Analysis of Algorithms

Assignment 01

Name : Sarita Sangrez

Roll Number : 231 - 2088

Section : CY-A-

Code 1.

Two nested loops (independent) $\rightarrow$ n times

$$\text{So, } O(n) = n^2$$

Code 2.

e.g. $n = 8$.

$$n = 8/2, 4/2, 2/2$$

$$n, n/2, n/4, n/8 \dots 1$$

$$n/2^k = 1$$

$$k = \log(n)$$

$$\text{So, } O(n) = \log(n)$$

Code 3.

outer loop : $n, n/2, n/4 \dots 1$

inner loop : j runs i times

$i = n$, inner does n

iterations

$i = n/2 \rightarrow$ iterations $n/2$ times

$$\text{So, } \sum_{k=0}^{\log_2 n} \frac{n}{2^k}$$

$$T(n) = n \left(1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{2^{\log_2 n}} \right)$$

$$O(n) = n \times \text{geometric series}$$

$$= \frac{1}{1-r}$$

Code 4.

statement runs constant

time. So, $O(n) = 1$.

Code 5.

outer loop $\rightarrow n$ times

inner loop $\rightarrow i$ times

$i = n$, inner does n iterations

$$n = 3, i = 1, 2, 3.$$

$$\text{So, } \sum_{i=1}^n i = 1 + 2 + \dots + n$$

$$O(n) = \frac{n(n+1)}{2} = O(n)^2$$

Date

Code 6.

outer loop $\rightarrow n$ times
inner loop $\rightarrow$ runs
infinite times as j is
always 0 (less than 5)
 $O(n) = \text{undefined}$.

Code 7

outer loop $\rightarrow n$ times
inner loop $\rightarrow n$ times
Total : $n \times n$
 $O(n) = n^2$

Code 8.

`continue()` skips the rest of
the loop and jumps to
next iteration

Here, `continue` increments
 i until $i = n-1$

`!` (int) gives ans 0.

"if $k \neq n$, break" runs
once and breaks so

$O(1)$.

Code 9.

loop runs once as false
 $n < 0$ fails when $n > 0$.
condition checked after the
body - as do...while

Code 10

function $\rightarrow O(n)$ times
main $\rightarrow O(n)$ times
has nested function
(++)
 $= O(n) \times O(n) \Rightarrow O(n^2)$

Code 11.

$i \leq n * n$

e.g. $n = 5$.

$i = 1, 2, 3, \dots$

$O(n) = n^2$

Code 12.

outer loop $\rightarrow n$ times

condition $j \% 1 == 0$ runs always.

inner loop runs exactly j
times so $O(j)$

Middle loop runs from 1
to $i * n$ with n loop cost $O(n)$

So total $\sum_{j=1}^{i*n} j = O((i*n)^2)$

outer loop $\rightarrow n$ and for each i ,
loop j does $(i*n)^2$ work

Date

Code 12 (continuation)

$$\text{Total work} : \sum_{i=1}^{n-1} (i \cdot n)^2 = n^2 \sum_{i=1}^{n-1} i^2 \quad \sum_{i=1}^{n-1} i^2 = O(n^3)$$

$$\text{So, } T(n) = n^2 \cdot O(n^3)$$

$$\text{e.g. } n=3 \quad \dots = O(n^5)$$

$i=1$

$i=2$

$i=3$

$j=1$ time, 2 time

$j=1$ time, 2, 3, 4, 5, 6

$k=1$ time, 2 time

$k=1, 1, 2, 2, 1, 2, 3, 4, 2, 3, 3, 1, 2, 3, 4, 5, 6$

Code 13.

Outer loop $\rightarrow n$ times

2nd outer loop $\rightarrow i \cdot n$ times $(\sum_{i=1}^{n+1} i)$

Inner loop $\rightarrow 1$ to j times $(O(j))$

$$T \text{ loop} : \sum_{i=1}^{n+1} j = O((n+1)^2)$$

$$1 \text{ loop} : \sum_{i=1}^n n^2 \cdot i^2 = n^2 \sum_{i=1}^n i^2 \quad \sum_{i=1}^n i^2 = n^3$$

$$= n^2 \times \frac{n^3}{3} = O(n^5)$$

Q2.

a) $f(n) \geq c \cdot g(n)$.

$$100n + 5 \geq cn^2 \quad \text{for } n \geq n_0$$

$\div$ by n : $100 + \frac{5}{n} \geq cn$ e.g. $n = 200$, $c = 1$

$$100.025 \not\geq 200$$

cn grows without bound as $n \rightarrow \infty$ while left side approaches 100. For large n , equality fails. So no such c, n_0 exist.

Date

b) $f(n) = \frac{n^2}{2} - \frac{n}{2}$

$$c_1 n^2 \leq \frac{n^2}{2} - \frac{n}{2} \leq c_2 n^2$$

$n \geq 1$
Upper Bound: $\frac{n^2}{2} - \frac{n}{2} \leq \frac{n^2}{2}$, so we take $c_2 = 1/2$

$n \geq 2$
Lower Bound: $\frac{n^2}{2} - \frac{n}{2} \geq \frac{n^2}{4}$ is false e.g. $n=2$, $\frac{4}{2} - \frac{2}{2} \geq \frac{4}{4} \Rightarrow 1 \geq 2$

Take $c_1 = 1/4$,

$$\frac{n^2}{2} - \frac{n}{2} \geq \frac{1}{4} n^2 \Rightarrow 2(n^2 - n) \geq n^2 \Rightarrow n(n-2) \geq 0$$

works for $n \geq 2$, $c_1 = 1/4$.

$$\text{So, } \frac{1}{4} n^2 \leq \frac{n^2}{2} - \frac{n}{2} \leq \frac{1}{2} n^2 \quad (n \geq 2)$$

$$f(n) = \Theta(n^2)$$

c) $6(n^3) \notin \Theta(n^2)$

$\Theta(n^2)$ means $O(n^2)$ and $\Omega(n^2)$ are true.

$$6n^3 \leq cn^2$$

$$6n \leq c$$

$6n$ grows without bound and right side is a constant.
It fails for Big O part so $6n^3 \notin \Theta(n^2)$

d) $f(n) = n \notin \Theta(\log n)$

check if $n \in \Omega(\log n)$

$$n \geq c \log_2 n \quad \text{for } n \geq 2 \rightarrow \text{True for } c_1 = 1, n_0 = 2$$

now check if $n \in O(\log n)$

$$n \leq c \cdot \log_2 n \quad n \geq n_0$$

Take $c = 10$, $n = 100$, $100 \leq 66.4 \rightarrow \text{False}$.

OR $\frac{n}{\log_2 n} \rightarrow \infty$ as $n \rightarrow \infty$ so can't be less than equal to c ($\frac{n}{\log_2 n} \leq c$)

Date

e) Identity : $\log_2(n^3) = 3 \log_2 n$
 $\log_2(n^3) = 3 \log_2 n \leq c \cdot \log_2 n$
 take $c=3$, $n_0=1$.

$$3 \cdot 0 \leq 3 \cdot 0 \Rightarrow \text{True}$$

Equality holds so $\log_2(n^3) \in O(\log_2 n)$

Q3.

start with $n/4$

1. $T = 3T(n/4) + n$

$$T(n/4) = 3T(n/16) + n/4$$

$$T = 3[3T(n/16) + n/4] + n$$

① $T = 9T(n/16) + \frac{3n}{4} + n$ $T(n/16) = 3T(n/64) + n/8$

$$T = 9[3T(n/64) + n/16] + \frac{3n}{4} + n$$

② $T = 27T(n/64) + \frac{9n}{16} + \frac{3n}{4} + n$

$$T = 3^k T(n/4^k) + n(1 + \frac{3}{4} + \frac{9}{16} + \dots + (\frac{3}{4})^{k-1})$$

$T(1) = 1$ so we stop when

$$\frac{n}{4^k} = 1 \quad k = \log_4 n$$

$$T = 3^{\log_4 n} \cdot T(1) + n \cdot \sum_{i=0}^{\log_4 n - 1} (\frac{3}{4})^i$$

$$3^{\log_4 n} = n^{\log_4 3} \rightarrow n^{0.792} \text{ dominates } 4n \cdot (1 - (3/4)^{\log_4 n}) \text{ constant harmonic}$$

$$T = n^{0.792} + 4n$$

$$T(n) = \Theta(n)$$

2. $T = 2 + T(n-1)$

$$T(n-1) = 2 + T(n-1-1)$$

① $T = 2 + 2 + T(n-2)$

$$T(n-1) = 2 + T(n-2)$$

$$T = 2 + 2 + 2 + T(n-3)$$

$$T(n-2) = 2 + T(n-3)$$

After k^{th} iterations,

$$T = 2k + T(n-k)$$

$$T = O(n)$$



Date

OR. $T(1) = 2 + T(0) = 2 + 2 = 4$

$T(2) = 2 + T(1) = 2 + 4 = 6$

$T(3) = 2 + T(2) = 2 + 6 = 8$

$T(n) = 2n + 2$

So $T(0) = 2 + 2 \cdot 0$

$T(k) = 2 + 2k$

$T(k+1) = 2 + 2(k+1) = 2 + 2k + 2$

$= 2 + T(k)$

$T = 2 + 2(k+1) \rightarrow$ holds for all n

3. $T(n) = T(n-1) + 1$

$T(n-1) = T(n-2) + 1$

$T = (T(n-2) + 1) + 1$

$T = T(n-2) + 2$

$T(n-2) = T(n-3) + 1$

$T = T(n-3) + 2 + 1$

$n-k=1 \quad T = T(n-3) + 3 \rightarrow T(1) + (n-1)$

$k=n-1 \leftarrow T = T(n-k) + k$

we know $T(1) = O(1)$, $T(1) = C$ constant

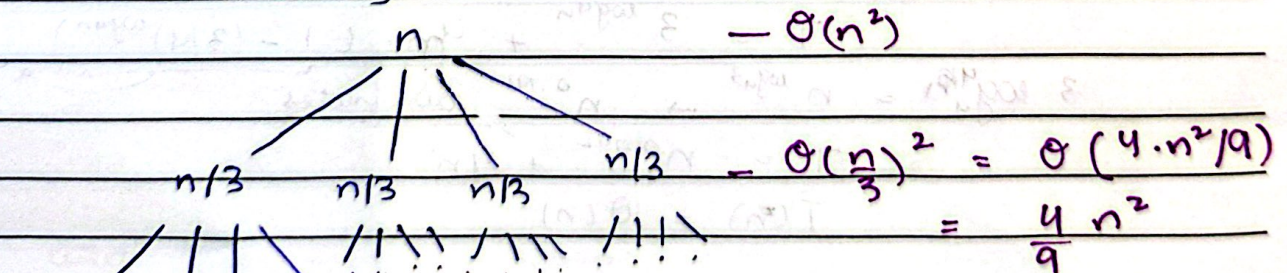
$T(n) = C + (n-1)$

$T(n) = n + (C-1)$

$T(n) = \Theta(n)$

Q3.

1. $T(n) = 4T(\frac{n}{3}) + \Theta(n^2)$



$\Theta(\frac{n}{9})^2 \times 16 = (\frac{4}{9})^2 n^2$

general: $\Theta((\frac{4}{9})^i n^2)$

$T = n^2 \left((\frac{4}{9})^0 + (\frac{4}{9})^1 + (\frac{4}{9})^2 + \dots + (\frac{4}{9})^k \right)$

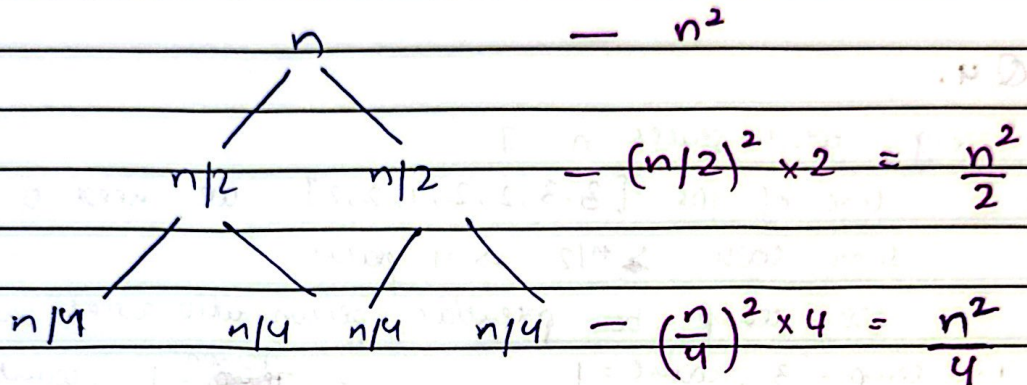
$T = n^2 \left(\frac{1}{1-r} \right)$

Kaaghaz Qalam

Date

$$T(n) = \Theta(n^2)$$

2. $T(n) = 2T(n/2) + n^2$



General: $\frac{n^2}{2^k}$, $k = \log_2 n$.

$$T(n) = \sum_{k=0}^{\log_2 n} \frac{n^2}{2^k} = n^2 \left(\frac{1}{2^0} + \frac{1}{2^1} + \frac{1}{2^2} + \dots + \frac{1}{2^k} \right)$$

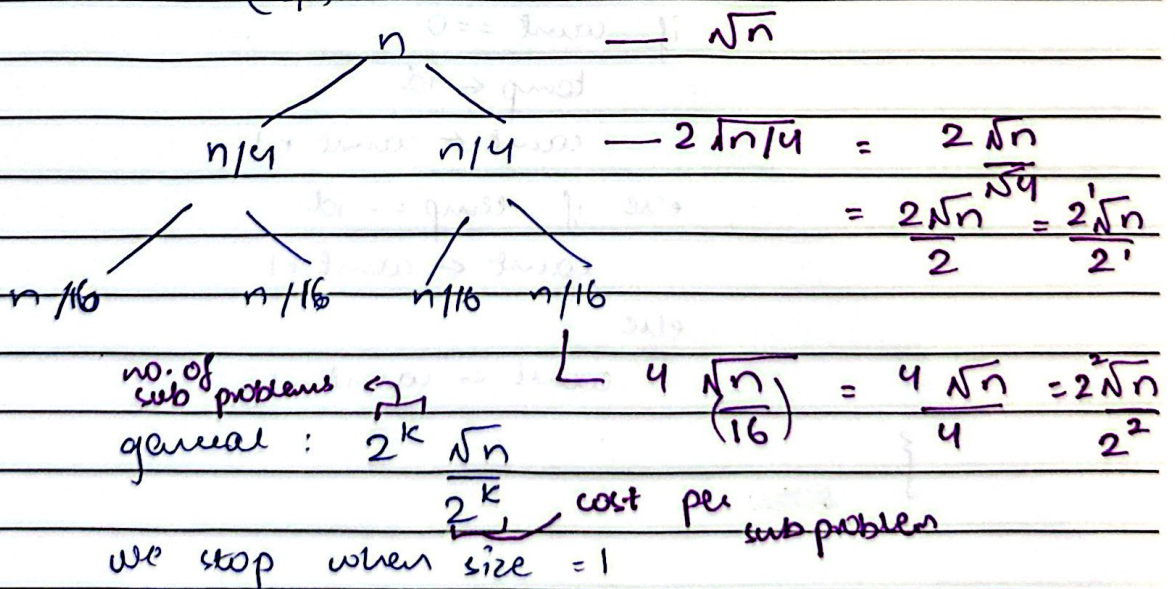
geometric series

$$= n^2 \left(\frac{1}{1-r} \right)$$

$$T(n) = \Theta(n^2)$$

3.

$$T(n) = 2T\left(\frac{n}{4}\right) + \sqrt{n}$$



Size of each sub problem. $\left\{ \frac{n}{4^k} = 1 \right\}$, $k = \log_4 n$

OR Height of Tree.

Total work of one level = $\sqrt{n}$.

Date

each level gives

no. of levels.

$$\text{So, } T(n) = \sqrt{n} \cdot \log_4 n$$

$$T(n) = O(\sqrt{n} \log n)$$

Q4.

e.g. total plates $n = 7$

line of IDs $[3, 3, 2, 2, 1, 2, 2]$. we need to find if some took $> n/2 \Rightarrow 4$ plates

let temp be potential person who takes most plates.

1. $\text{temp} = 3$, $\text{count} = 1$

5. $\text{Person} = 1$, $\text{count} = 0$, so $\text{temp} = 1$
 $\text{count} = 1$

2. $\text{temp} = 3$, $\text{count} = 2$

6. $\text{temp} = 2 \neq 1$, $\text{count} = 0$, so $\text{temp} = 2$,
 $\text{count} = 1$

3. $\text{temp} = 2 \neq 3$, $\text{count} = 1$

7. $\text{count} = 2$, $\text{count} = 1$.

4. $\text{temp} = 2 \neq 3$, $\text{count} = 0$

Pseudocode :

FindMajority_Person (array of ID , n) {

temp $\leftarrow$ none.

count $\leftarrow 0$

for each id in array of ID

if count == 0

temp $\leftarrow$ id

count $\leftarrow$ count + 1

else if temp == id

count $\leftarrow$ count + 1

else

count $\leftarrow$ count - 1

}